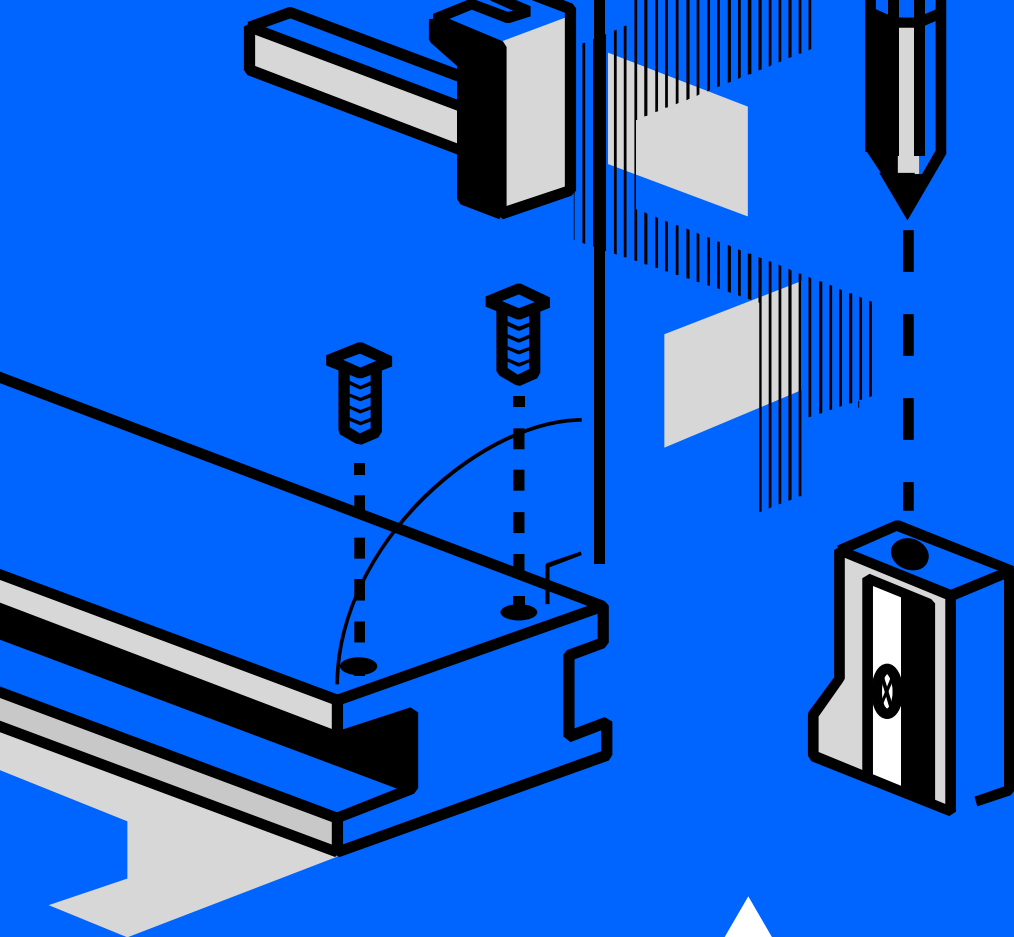




רענון JS מודרני

המכללה
הטכנולוגית
באר שבע

בשיתוף WORLD ORT
קדימה מדע



למה אנחנו כאן?

בקורס שלנו, אנחנו כל הזמן:

- נקבל בקשות (Requests) עם מידע.
- נצטרך לשלוח מידע מבסיס נתונים (DB).
- נעבד את המידע הזה (לסנן, לשנות מבנה, לסכם).
- נחזיר אותו כ-Response.

המתודות שנרענן היום הן **90% מהלוגיקה** שתכתבו בשרת.

ES Modules (הבסיס המודרני)

אנחנו משתמשים ב-`import` (במקום `require`). ישנן שתי דרכים עיקריות לייבא קוד:

2. ייבוא הכל כיחידה אחת (Namespace Import)

- מייבאים את כל מה שהקובץ מייצא (export) לתוך אובייקט אחד.

```
// main.js
import * as utils from './helpers.js';

// עכשיו ניגש לפונקציות דרך האובייקט
utils.funcA();
utils.funcB();
```

1. ייבוא חלקי (Named Exports)

- מייבאים רק פונקציות או משתנים ספציפיים שצריך, באמצעות סוגריים מסולסלים.
- זהו השימוש הנפוץ ביותר.

```
// helpers.js
export const funcA = () => { /* ... */ };
export const funcB = () => { /* ... */ };

// main.js
import { funcA } from './helpers.js';
```

ייבוא JSON (מודרני):

```
import users from './users.json' assert { type: 'json' };
```

map, filter, forEach

המתודות הכי נפוצות. כולן לא משנות את המערך המקורי.

- **map()** - "יוצר מערך חדש מאותו גודל, עם שינוי על כל איבר."
- **forEach()** - "פשוט רץ על כל איברי המערך. לא מחזיר כלום."

```
const fruits = ['Banana', 'Apple'];
fruits.forEach(fruit => console.log(fruit));
// ידפיס:
// Banana
// Apple
```

```
const numbers = [1, 2, 3];
const doubled = numbers.map(num => num * 2);
// יהיה: doubled [2, 4, 6]
```

- **filter()** - "יוצר מערך חדש (קטן יותר) רק עם האיברים שעברו תנאי."

```
const numbers = [1, 2, 3, 4];
const evens = numbers.filter(num => num % 2 === 0);
// יהיה: evens [2, 4]
```

reduce

המתודה החזקה ביותר. "מקפלת" מערך שלם לערך אחד (מספר, אובייקט וכו').

דוגמה: סיכום סך כל ההזמנות.

```
const orders = [  
  { orderId: 1, total: 1548 },  
  { orderId: 2, total: 448 },  
  { orderId: 3, total: 2523 },  
];  
  
// 'sum' הוא המצטבר, 0 הוא הערך ההתחלתי  
const totalSum = orders.reduce((sum, currentOrder) => {  
  return sum + currentOrder.total;  
}, 0);  
  
// totalSum 4519 יהיה
```

Mutators

המתודות האלו משנות את המערך המקורי.

- **splice()** - "האולר השוויצרי": מוסיף, מוחק או מחליף איברים במקום.
- **sort()** - "ממין את המערך במקום. חובה לספק פונקציית השוואה למספרים!"

```
const numis = [6, 2, 7, 1];  
numis.sort((a, b) => a - b); // מיון בסדר עולה  
// numis [1, 2, 6, 7] עכשיו:
```

```
const fruits = ['Banana', 'Apple', 'Cherry'];  
// "Orange" - החלף מאינדקס 1, פריט 1, ב  
fruits.splice(1, 1, "Orange");  
// fruits עכשיו: ["Banana", "Orange", "Cherry"]
```

Async - תכנות אסינכרוני



הבעיה: פעולות בשרת (קריאה מ-DB, פנייה ל-API) **לוקחות זמן**. אנחנו לא רוצים "לתקוע" את כל התוכנית בזמן שאנחנו מחכים.

הפתרון: `await` / `async` (או: אנלוגיית הפיצה)

דמיינו שהזמנתם פיצה (פעולה שלוקחת זמן):

- הדרך ה"תוקעת" (סינכרונית): להזמין ולשבת ליד הדלת 40 דקות, בלי לזוז, עד שהשליח יגיע. בזבזתם את כל הזמן הזה.
- הדרך ה"אסינכרונית" (`async`): להזמין את הפיצה, וברגע שההזמנה נקלטה, אתם חופשיים! אתם הולכים להתקלח, לראות טלוויזיה, וכו' (שאר הקוד ממשיך לרוץ).

כשהשליח מגיע (ה"הבטחה" מתקיימת), אתם עוצרים רגע את המקלחת, הולכים לקבל את הפיצה, וחוזרים להמשיך בערב שלכם.

ההגדרות הטכניות:

- `async`: מילה ששמים לפני פונקציה (כמו "תהליך הזמנת הפיצה"). הופכת אותה לכזו שמחזירה "הבטחה" (Promise).
- `await`: מילה ששמים בתוך פונקציית `async` (כמו "לחכות לשליח").
 - מה זה עושה? זה משהה את הריצה רק של הפונקציה הנוכחית (אתם מפסיקים לחכות לפיצה), משחרר את הת'רד הראשי (אתם פנויים להתקלח), וממשיך בביצוע שאר הקוד.

תרגיל Async פשוט

```
// 1. פונקציה שמדמה "עבודה קשה" (לוקחת שנייה)  
async function doHeavyWork() {  
  
    console.log("    LOG B: מתחיל עבודה קשה...");  
  
    // doHeavyWork הזה משהה *רק* את הפונקציה await-ה  
    await new Promise(resolve => setTimeout(resolve, 1000));  
  
    console.log("    LOG C: ...סיים עבודה קשה...");  
}  
  
// --- תחילת הריצה ---  
  
console.log("LOG A: התוכנית מתחיל את...");  
  
doHeavyWork(); // 🚀 מפעיל את הפונקציה האסינכרונית  
  
console.log("LOG D: !התוכנית סיימה");
```

קוד:

בואו נכתוב קובץ פשוט ונראה מה קורה.

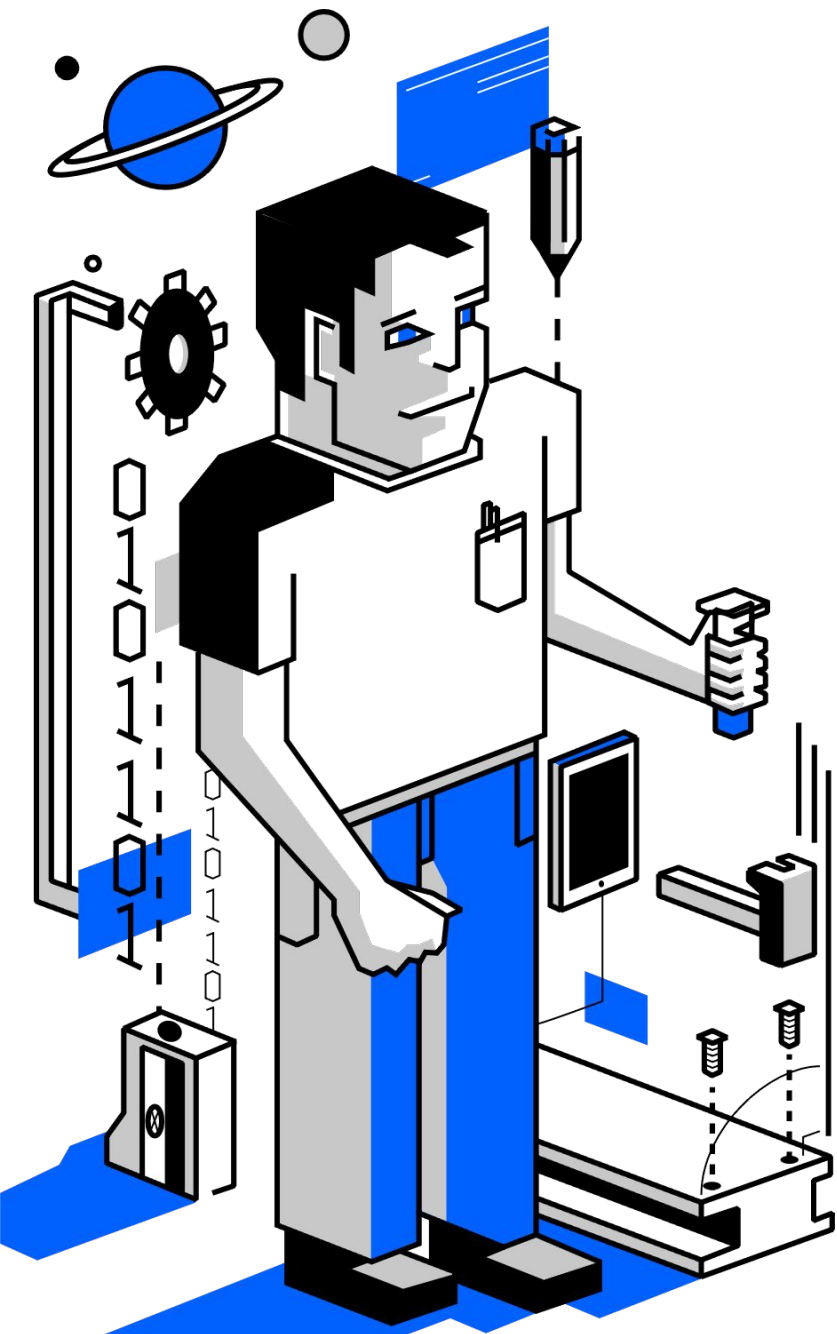
מה יודפס בקונסול?

סדר הריצה האמיתי:

1. התוכנית מתחילה ומדפיסה: **LOG A: מתחיל את התוכנית.**
2. התוכנית קוראת ל-`doHeavyWork()`.
3. הפונקציה `doHeavyWork` מתחילה ומדפיסה: **LOG B: מתחיל עבודה קשה...**
4. הפונקציה מגיעה ל-`await` ומושהית. השליטה חוזרת לתוכנית הראשית.
5. התוכנית הראשית ממשיכה לרוץ ומדפיסה: **LOG D: התוכנית סיימה!**
6. (עוברת שנייה...)
7. ה-`Promise` מסתיים.
8. הפונקציה `doHeavyWork()` חוזרת לפעול ומדפיסה: **LOG C: ...סיים עבודה קשה.**

תרגול

כנסו למודל הורידו ופתחו את הקובץ `js.workshop.js`.



המכללה
הטכנולוגית
באר שבע

בשיתוף WORLD ORT
קדימה מדע



שאלות